

Monitoring de systèmes hydrauliques

Pipeline MLOps — De l'entraînement à la production

Lina Rhiati Hazime Mohammed Horache Benjamin Lepourtois Grégoire Petit



DATA713 — Mars 2026

Cas d'usage : Maintenance prédictive industrielle

Dataset : UCI — Condition Monitoring of Hydraulic Systems

- 2205 cycles $\times$ 17 capteurs
- 4 états de composants à prédire simultanément
- Labels issus de `profile.txt` (supervisé)

Formulation : Classification multi-classe multi-output

→ Pourquoi pas détection d'anomalies ? Données labellisées

→ diagnostic par composant plus actionnable

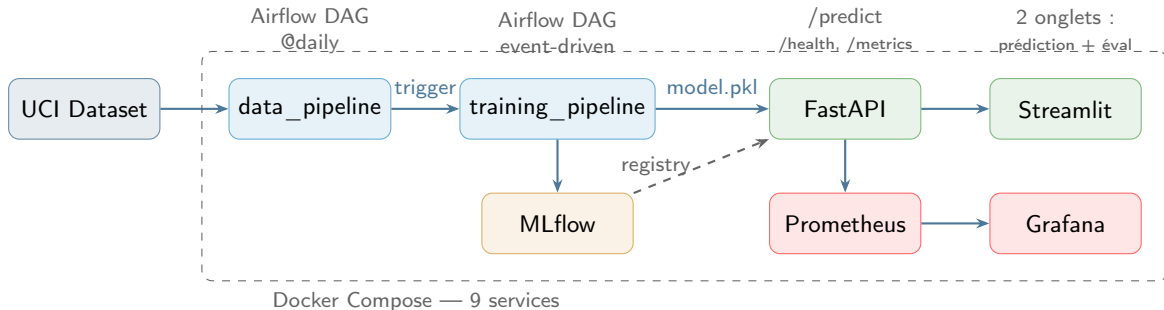
Cible	Composant	Classes
cooler	Refroidisseur	3
valve	Vanne	4
pump	Pompe	3
accumulator	Accumulateur	4

17 capteurs :

PS1–PS6, EPS1, FS1–FS2,
TS1–TS4, VS1, CE, CP, SE

Feature eng. : moyenne par cycle

Vue d'ensemble de l'architecture



Pourquoi Docker Compose ?

Un seul docker compose up → 9 services.
Reproductible, aucune installation manuelle.

Pourquoi des manifests K8s ?

Pipeline CD prêt pour la production. Déploiement conditionnel au secret KUBECONFIG.

Pipeline ML & Continuous Training

Modèle : MultiOutputClassifier(RF)

Pourquoi ? Un seul entraînement, features cohérentes, déploiement plus simple que 4 modèles

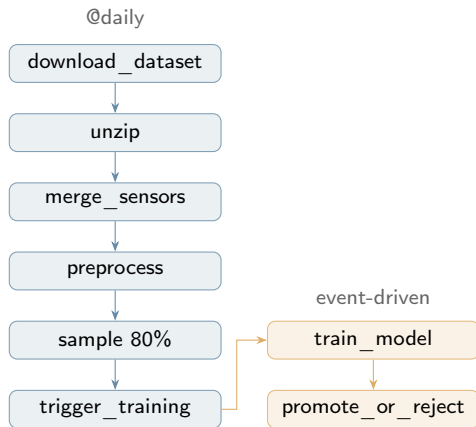
Résultats (test set 20%) :

Cible	Acc.	F1 _{macro}	F1 _{w.}
Cooler	1.000	1.000	1.000
Valve	0.948	0.947	0.948
Pump	0.993	0.993	0.993
Accumulator	0.990	0.990	0.990

Choix de conception :

- F1 macro pour la comparaison (gère le déséquilibre)
- DAG délègue à src/train.py (séparation des responsabilités)
- Promotion seulement si $F1_{new} > F1_{prod}$

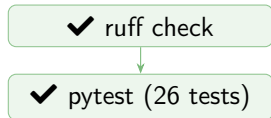
Logique CT (2 DAGs) :



Pourquoi échantillonner 80% ?

Dataset statique → l'échantillonnage simule

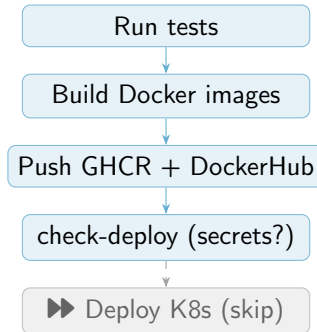
CI (chaque push & PR)



26 tests en 3 suites :

Suite	#	Vérifie
DAGs	9	structure, task IDs, deps
Model	7	entraînement, F1, features
Preproc.	10	merge, filtre, NaN

CD (push sur main)



Double registry : GHCR + DockerHub (redondance)

Tags : latest + git SHA → rollback possible

Gestionnaire : uv (rapide, lockfile, reproducible)

FastAPI

- `POST /predict`
Validation Pydantic
17 capteurs → 4 états
- `GET /health`
Sonde K8s readiness
- `GET /metrics`
Scraping Prometheus
- `GET /docs`
Swagger auto-généré

Streamlit

Onglet 1 — Prédiction

- 17 champs capteurs
- Appel API → 4 états

Onglet 2 — Évaluation

- Métriques par cible
- Matrices de confusion
- Rapports de classification

Via `model_metrics.json`

→ Captures d'écran dans les slides backup

Monitoring

- **Prometheus**
Scrape `/metrics`
+ Node Exporter
- **Grafana**
Dashboard auto-provisionné
au compose up
- **Alerting**
Callbacks email Airflow
en cas d'échec DAG

Organisation de l'équipe :

Qui	Périmètre
Lina	Données & pipeline ML, MLflow
Grégoire	Airflow, CT, CI/CD, tests, docs
Benjamin	API FastAPI, webapp Streamlit
Mohammed	Docker, K8s, CD, monitoring

Workflow Git :

- Feature branches → PR → main
- CI obligatoire sur chaque PR
- 20+ PRs, conventional commits
- Reviews Copilot

Limites assumées :

- Dataset statique — CT conçu pour la prod mais validé sur données échantillonnées
- Pas de cluster K8s — pipeline CD prêt, deploy skip
- API charge `model.pkl` localement (en prod : requête au MLflow Registry)
- Imports lourds dans les DAGs → différés dans les fonctions (limite 30s parsing)

Principe clé de conception :

Séparation des responsabilités :

`src/` = logique métier, `dags/` = orchestration,

`api/` = serving, `webapp/` = IHM

Couverture des objectifs

10/10 objectifs requis :

#	Objectif	Statut
1	Extraction & prétraitement des données	✓
2	Modèle ML	✓
3	Model registry (MLflow)	✓
4	Pipeline Airflow de réentraînement	✓
5	MLflow experiment tracking	✓
6	API (FastAPI + Swagger)	✓
7	WebApp (Streamlit, 2 onglets)	✓
8	Continuous Training (CT)	✓
9	Docker + K8s + CI/CD	✓
10	Versionnage GitHub & docs	✓

Bonus réalisés (3/8) :

#	Bonus	
12	Monitoring (Prom+Grafana)	✓
14	Versionnage modèle / rollback	✓
17	Alerting email	✓
11	Données temps réel	✗
13	Test de charge (Locust)	✗
15	Human in the loop	✗
16	Gestion accès API	✗

Stack :

Python 3.11 · scikit-learn · MLflow
Airflow · FastAPI · Streamlit
Docker · K8s · GitHub Actions
Prometheus · Grafana · uv



SCREENSHOT

Liste des DAGs Airflow
(data_pipeline + training_pipeline)

DAGs

Vue liste des



SCREENSHOT

Graphe data_pipeline
(6 tâches chaînées)

tâches

Vue graphe des



SCREENSHOT

Runs MLflow
(métriques F1 par cible)

expériences

Suivi des



SCREENSHOT

MLflow Model Registry
(stages Production / Archived)

modèles

Versionnage des

 SCREENSHOT

Swagger /docs
(schéma Pydantic)

FastAPI

 SCREENSHOT

Streamlit Onglet 1
(formulaire prédiction)

Prédiction

 SCREENSHOT

Streamlit Onglet 2
(matrices de confusion)

Évaluation

Backup — Monitoring & GitHub



SCREENSHOT

Dashboard Grafana
(métriques API + système)

Monitoring



SCREENSHOT

PRs GitHub
(20+ mergées, CI gates)

Collaboration



SCREENSHOT

GitHub Actions CI
(ruff + pytest vert)



SCREENSHOT

Pipeline CD
(build + push + check-deploy)